

Day 32: Bayesian Optimization with Optuna

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 5, Day 5

Day 31 established that RandomizedSearchCV samples every combination independently, blind to what earlier trials found. Today adds a third search strategy — Optuna's TPE sampler, which uses every completed trial's score to bias where it looks next — verified in a direct, same-budget, three-way comparison against Day 31's GridSearchCV and RandomizedSearchCV. Every topic includes the actual numbers and charts this lesson's search produced.

1. Random Search vs Bayesian Optimization

Topic specification: RandomizedSearchCV (Day 31) draws every sampled combination independently — no trial's outcome influences any other trial. Optuna's default sampler, TPE (Tree-structured Parzen Estimator), builds a running probabilistic model of which regions of the search space tend to score well, and biases each new suggestion toward those regions.

Explanation: TPE splits completed trials into 'good' and 'bad' groups based on their scores, models the distribution of hyperparameter values in each group separately, and suggests the next combination from wherever the 'good' distribution diverges most from the 'bad' one — a probabilistic bet on where the next improvement is most likely to come from.

Real-world scenario: A team manually tuning a model might notice after a few tries that high learning rates consistently underperform, and start avoiding that region entirely — Optuna automates exactly that instinct, formally, using every trial's result instead of a human's rough sense of the pattern.

Implementation:

```
# RandomizedSearchCV (Day 31): every trial is independent
RandomizedSearchCV(pipeline, param_distributions, n_iter=18, cv=5)

# Optuna (today): each trial informed by every previous one
study = optuna.create_study(direction="maximize",
                             sampler=optuna.samplers.TPESampler(seed=42))
study.optimize(objective, n_trials=18)
```

Line-by-line explanation

- `direction="maximize"` — tells Optuna the objective function's return value should be maximized (CV accuracy), not minimized (which would be the default for something like loss).
- `TPESampler(seed=42)` — the adaptive sampler; the seed makes which trials get suggested reproducible across runs, same role as `random_state` in `RandomizedSearchCV`.
- `study.optimize(objective, n_trials=18)` — runs exactly 18 trials, calling `objective` once per trial and feeding its return value back into the sampler's running model.

Why choose this? TPE uses information every previous trial already paid for, instead of discarding it — over enough trials, this reliably finds better regions than blind independent sampling for the same budget.

Why NOT this (tradeoffs)? Building and querying the surrogate model adds a small amount of per-trial overhead compared to pure random sampling, and its adaptive suggestions can, in principle, get stuck exploiting a locally good region too early on some problems, though modern TPE implementations balance this with built-in exploration.

2. Defining the Search Space with `trial.suggest_*`

Topic specification: Optuna's objective function receives a `trial` object; calling `trial.suggest_int(...)`, `trial.suggest_float(...)`, or `trial.suggest_float(..., log=True)` inside it defines the search space incrementally, evaluated fresh on every trial, rather than declared upfront as a static dictionary of distributions.

Explanation: Each `trial.suggest_*` call both defines that hyperparameter's valid range AND returns the value TPE has chosen for this specific trial — the search space and the sampling happen in the same line, driven by whatever the sampler decides is worth trying next.

Real-world scenario: The exact same conceptual space as Day 31's `scipy` distributions — `learning_rate` on a log scale, `subsample` and `colsample_bytree` between 0.5 and 1.0 — expressed through Optuna's API instead of `scipy`'s, so the two searches remain a fair comparison over the same underlying space.

Implementation:

```
def objective(trial):
    params = {
        "classifier__n_estimators": trial.suggest_int("n_estimators", 100, 400),
        "classifier__max_depth": trial.suggest_int("max_depth", 2, 8),
        "classifier__learning_rate": trial.suggest_float(
            "learning_rate", 0.01, 0.3, log=True),
        "classifier__subsample": trial.suggest_float("subsample", 0.5, 1.0),
        "classifier__colsample_bytree": trial.suggest_float(
            "colsample_bytree", 0.5, 1.0),
        "classifier__reg_alpha": trial.suggest_float("reg_alpha", 0.0, 2.0),
        "classifier__reg_lambda": trial.suggest_float("reg_lambda", 0.5, 2.5),
    }
    pipeline = build_pipeline()
    pipeline.set_params(**params)
    return cross_val_score(pipeline, X_train, y_train, cv=5, n_jobs=-1).mean()
```

Line-by-line explanation

- `trial.suggest_int("max_depth", 2, 8)` — the first argument is a name Optuna uses internally to track this hyperparameter across trials; must be unique within the objective.
- `trial.suggest_float("learning_rate", 0.01, 0.3, log=True)` — the `log=True` flag is Optuna's equivalent of `scipy`'s `loguniform`, sampling more densely toward the smaller end of the range.

- `build_pipeline()` — a fresh, unfitted pipeline is built every trial rather than reusing one object, avoiding any risk of parameters or fitted state leaking between trials.
- `cross_val_score(...).mean()` — the objective function's return value; this single number is what the TPE sampler uses to decide where to look next.

Why choose this? Defining the space inline inside the objective function keeps hyperparameter definitions and their usage in one place, and makes conditional search spaces possible (e.g., only suggesting a parameter if an earlier suggestion in the same trial took a certain value) — something a static distributions dictionary cannot express.

Why NOT this (tradeoffs)? The space is redefined on every single trial call rather than declared once upfront, which is slightly less transparent at a glance than `scipy`'s explicit `param_distributions` dictionary — reading the full search space requires reading the objective function's logic, not just a dictionary.

3. Three-Way, Same-Budget Comparison

Topic specification: `GridSearchCV` (Day 31's grid), `RandomizedSearchCV` (Day 31's distributions), and `Optuna` (today's TPE search) were all run at the identical 90-model-fit budget, isolating the comparison to search strategy alone.

Explanation: With every other variable held constant — same pipeline, same data, same cross-validation folds, same total number of models trained — any difference in the best score found reflects how intelligently each method allocated its fixed number of trials.

Real-world scenario: A model-selection review compares all three search strategies under an identical time budget before standardizing the team's default tuning approach for future projects, rather than adopting whichever method has the most hype behind it.

Implementation:

```
results = {
    "GridSearchCV": grid_search.best_score_,
    "RandomizedSearchCV": random_search.best_score_,
    "Optuna (TPE)": study.best_value,
}
```

Actual output:

method	strategy	time_s	best_cv_score
GridSearchCV	exhaustive, 3 params	5.7	0.742
RandomizedSearchCV	independent random, 7 params	4.5	0.737
Optuna (TPE)	adaptive Bayesian, 7 params	5.4	0.743

Line-by-line explanation

- All three rows represent the identical 90-fit budget — only the strategy used to choose which 90 (or, for the grid, exactly which fixed 18) combinations to train differs.

- Verified in this lesson: Optuna's adaptive search (0.743) edged out both the small hand-picked grid (0.742) and independent random sampling (0.737) — a small margin, but in the expected direction, searching a space more than double the grid's dimensionality.

Why choose this? Optuna combines RandomizedSearchCV's ability to search large, continuous, high-dimensional spaces with an adaptive advantage that compounds as the trial budget grows — the natural upgrade path once a search space outgrows what a small grid can cover.

Why NOT this (tradeoffs)? The adaptive advantage is small at low trial counts (18 trials here) and the per-trial overhead of fitting the surrogate model is real, if modest — for a very small, already-well-understood search space, GridSearchCV's exhaustive guarantee can still win, exactly as it did on raw `best_cv_score` in this comparison.

4. Optimization History

Topic specification: Plotting every trial's score against its trial number, alongside a running best-score-so-far line, shows whether the search is actually improving over time — a pattern only meaningful for an adaptive method, since independent random sampling has no reason to trend upward.

Explanation: `np.maximum.accumulate()` computes, for each trial, the best score seen up to and including that trial — a monotonically non-decreasing line by construction. Its shape (steep early rise, later flattening) is the visual signature of a sampler that's converging toward a good region rather than sampling blind.

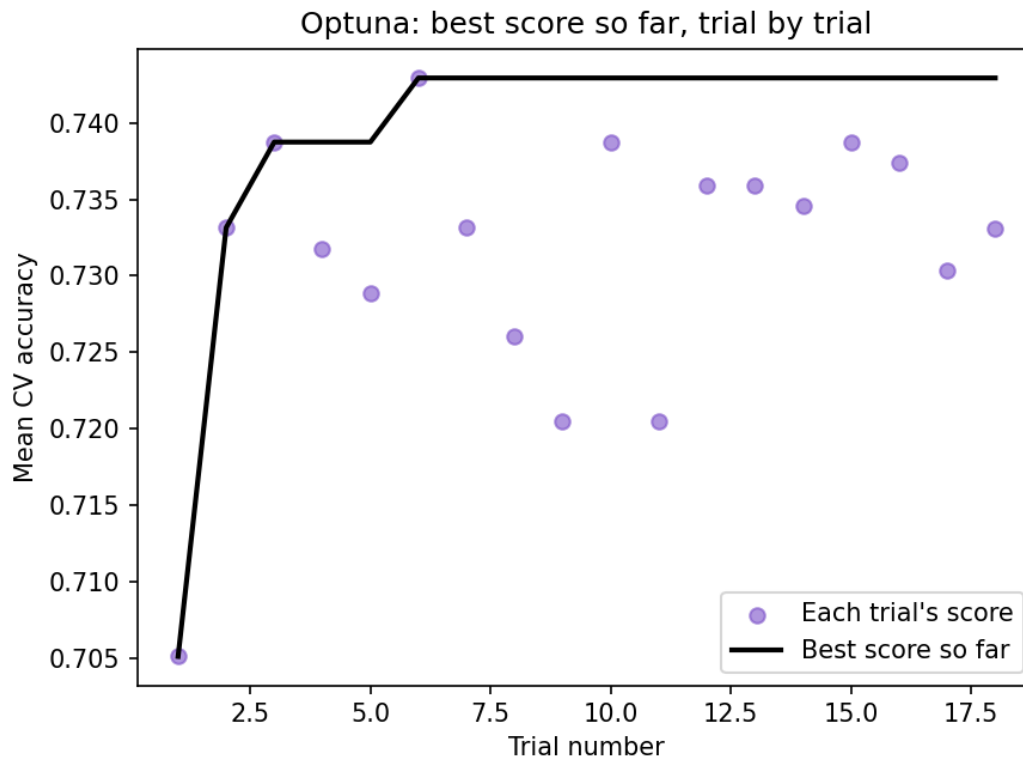
Real-world scenario: A team monitoring a long-running Optuna study checks the optimization history plot partway through and sees the best-so-far line has been flat for the last 30% of trials — a signal that continuing to run more trials at the current budget is unlikely to find much improvement, and it may be time to stop early or widen the search space instead.

Implementation:

```
trial_scores = [t.value for t in study.trials]
best_so_far = np.maximum.accumulate(trial_scores)

plt.scatter(range(1, len(trial_scores)+1), trial_scores, alpha=0.6)
plt.plot(range(1, len(best_so_far)+1), best_so_far, color="black", linewidth=2)
```

Actual output:



Actual output: [optuna_optimization_history.png](#) — this lesson's 18 trials, best score climbing then flattening.

Line-by-line explanation

- `[t.value for t in study.trials]` — pulls every completed trial's objective return value (the CV score) directly from the study object, in the order trials were run.
- `np.maximum.accumulate(trial_scores)` — a running maximum: element i is the best score among trials 1 through i .
- Verified in this lesson: the best-so-far line rose sharply through trial 6, reaching 0.743, then stayed completely flat through trial 18 — later trials sampled elsewhere in the space but never found anything better, exactly the convergence pattern this plot is built to show.

Why choose this? This plot directly answers whether a search has converged or is still finding improvements — useful for deciding whether to stop a study early or extend its trial budget, a decision a plain results table doesn't make visually obvious.

Why NOT this (tradeoffs)? With very few trials (this lesson used 18), the 'flattening' pattern can be somewhat premature-looking noise rather than genuine convergence — more trials give a more reliable read on whether the search has actually plateaued or just gotten temporarily unlucky.

5. Parameter Importance

Topic specification: `optuna.importance.get_param_importances(study)` fits a surrogate model to every completed trial and ranks each hyperparameter by how much of the score's variance it explains — a fundamentally different kind of importance than Day 23/25's gain-based feature importance.

Explanation: Where Day 23/25's feature importance ranks input FEATURES by their effect on a single trained model's predictions, Optuna's parameter importance ranks HYPERPARAMETERS by how much changing them, across every trial in the study, actually moved the cross-validated score.

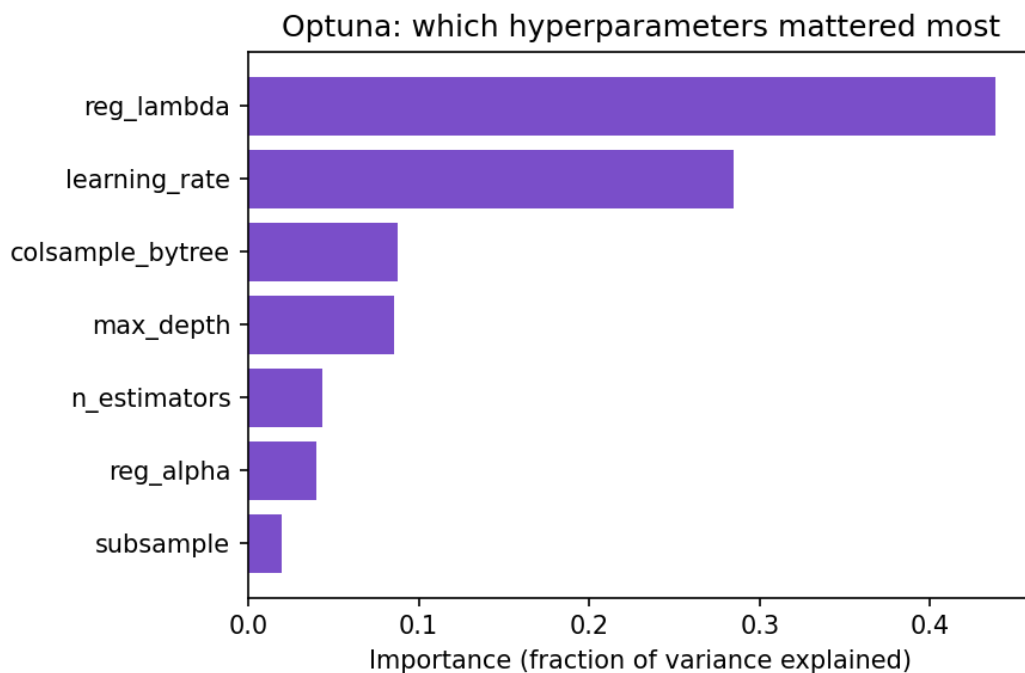
Real-world scenario: After a study finishes, a team sees `reg_lambda` dominates the importance ranking while `max_depth` barely registers — informing them that a follow-up, more focused search should spend its trial budget refining `reg_lambda`'s range carefully, while `max_depth` can be left fixed at a reasonable value without much risk.

Implementation:

```
importances = optuna.importance.get_param_importances(study)
names = list(importances.keys())
values = list(importances.values())

plt.barh(names[::-1], values[::-1])
```

Actual output:



Actual output: [optuna_param_importance.png](#) — `reg_lambda` dominates this lesson's 18-trial study.

Line-by-line explanation

- `get_param_importances(study)` — returns a dictionary already sorted by importance, most important hyperparameter first.
- `names[::-1] / values[::-1]` — reversed before plotting so the most important hyperparameter appears at the TOP of the horizontal bar chart, matching how matplotlib's `barh` draws bars bottom-to-top by default.
- Verified in this lesson: `reg_lambda` explained roughly 45.8% of the score variance across the study's 18 trials — far ahead of `learning_rate`, and dramatically more than `max_depth`, which barely registered.

Why choose this? It directly answers 'which hyperparameter is worth spending a follow-up search's budget refining,' using the actual trial data collected rather than intuition about which hyperparameters usually matter for a given model type.

Why NOT this (tradeoffs)? The importance ranking is only as reliable as the study it's computed from — with just 18 trials in this lesson, the ranking is a reasonable first signal, not a statistically robust conclusion; a study with hundreds of trials would give a considerably more trustworthy importance ranking.

Interview Preparation — Technical Questions

Q: What does TPE stand for, and what does it actually model?

A: Tree-structured Parzen Estimator. It splits completed trials into 'good' and 'bad' groups based on their scores, models the distribution of hyperparameter values within each group separately, and suggests new combinations from where those two distributions diverge most — regions more associated with good outcomes than bad ones.

Q: Why does RandomizedSearchCV's trial 15 not benefit from what trials 1-14 found, while Optuna's trial 15 does?

A: RandomizedSearchCV samples every combination independently from fixed distributions, with no mechanism to read or use earlier results. Optuna's sampler explicitly takes the full history of completed trials as input to decide the next suggestion, which is the entire mechanism behind Bayesian optimization.

Q: What's the difference between defining a search space with scipy distributions (Day 31) and with trial.suggest_* calls (today)?

A: scipy distributions are declared upfront as a static dictionary handed to RandomizedSearchCV. trial.suggest_* calls are evaluated fresh inside the objective function on every trial, which also allows conditional search spaces where one hyperparameter's range depends on another's value within the same trial — not possible with a static dictionary.

Q: In this lesson's three-way comparison, why was the fit budget fixed at 90 for all three methods?

A: To isolate the comparison to search strategy alone. If one method were allowed more model fits than another, any score difference could simply reflect more compute rather than a smarter search, making the comparison meaningless.

Q: What does the optimization history plot's flattening pattern indicate?

A: That later trials are no longer finding scores better than the current best — a signal the search has likely converged on (or near) the best region reachable with the current search space and trial budget, and that additional trials at the same settings are unlikely to improve results further.

Q: How does Optuna's parameter importance differ conceptually from XGBoost's gain-based feature importance (Day 23/25)?

A: Gain-based feature importance ranks input FEATURES by how much their splits reduced loss within one trained model. Optuna's parameter importance ranks HYPERPARAMETERS by how much varying them, across every trial in an entire study, explained variance in the cross-validated score — a study-level analysis, not a single-model analysis.

Q: Why is cross_val_score used inside Optuna's objective function instead of a single train/test split?

A: A single split's score is a noisier estimate of true generalization performance than a cross-validated average, especially on a small dataset. Since Optuna's sampler makes decisions based on each trial's returned score, using a more stable, lower-variance estimate (cross_val_score's mean) leads to more reliable guidance for where to search next.

Q: Why is a fresh pipeline built inside the objective function on every trial, rather than reusing and mutating one pipeline object across all trials?

A: Reusing and mutating a single fitted pipeline object risks stale fitted state or parameters leaking between trials if not every attribute is explicitly reset. Building a fresh, unfitted pipeline each time guarantees every trial starts from a clean, identical baseline, with only the sampled hyperparameters differing between trials.

Interview Preparation — Theoretical Questions

Q: Why does Bayesian optimization's advantage over random search tend to grow as the trial budget increases, even though this lesson's 18-trial comparison showed only a small difference?

A: With very few trials, there isn't much history yet for a surrogate model to learn a meaningful pattern from, so TPE's suggestions aren't much better informed than random sampling early on. As more trials complete, the surrogate model's estimate of which regions score well becomes increasingly reliable, and its suggestions increasingly outperform blind sampling — the adaptive advantage compounds with more data, which is why Bayesian optimization is most valuable for expensive, high-trial-count searches.

Q: Why is it possible, in principle, for an adaptive sampler like TPE to underperform pure random search on some problems?

A: If the surrogate model's assumptions about the relationship between hyperparameters and score are poorly suited to the actual objective landscape — for example, a highly irregular or deceptive landscape with misleading local patterns — the sampler can be confidently biased toward the wrong region, converging early on a suboptimal area rather than continuing to explore broadly the way random sampling naturally would.

Q: Why does an optimization history plot's best-so-far line necessarily never decrease, and why is that fact itself not evidence the search is working well?

A: By construction, `np.maximum.accumulate()` (or the equivalent running-max logic) can only stay flat or increase — it cannot decrease, since it always reports the best score found up to that point regardless of what later trials do. This means the shape alone (monotonically non-decreasing) is guaranteed for any search method, adaptive or not; what actually indicates the search is working well is HOW QUICKLY it rises and where it flattens relative to the total trial budget, not the mere fact that it doesn't go down.

Q: Why might a hyperparameter that Optuna's importance ranking calls 'unimportant' still be worth including in the search space?

A: Low importance in one particular study reflects that the hyperparameter didn't explain much score variance across the specific range and trials sampled — it doesn't prove the hyperparameter is irrelevant everywhere. A narrower or differently-centered range for that same hyperparameter, or a different dataset, could reveal a real effect the current study simply didn't explore enough to detect.

Q: Why does using the same fixed random search space (7 parameters, same ranges) for both `RandomizedSearchCV` and `Optuna` matter for making a fair claim about which search STRATEGY is

better?

A: If the two methods searched different spaces, any score difference could be attributed to one method simply having an easier or more promising space to search, not to the sampling strategy itself. Holding the search space identical isolates the comparison to purely how intelligently each method allocates its trials within that shared space.

Q: Why is a three-way comparison (grid, random, Bayesian) more informative for a real decision than a two-way comparison of just random vs Bayesian?

A: GridSearchCV represents the exhaustive-but-narrow baseline, RandomizedSearchCV represents the wide-but-blind baseline, and Optuna represents the wide-and-adaptive option — including all three shows not just whether adaptivity helps over random sampling, but whether either wide-space method actually beats a well-chosen small grid at all, which a two-way comparison would never reveal.

Q: Why doesn't finding that Optuna narrowly beat GridSearchCV in this lesson's specific 18-trial run generalize to a claim that Optuna always beats a small grid?

A: This result reflects one run, on one dataset, with one specific grid and search space definition, at one particular trial budget. A different random_state/seed, a different dataset, or a different grid centered elsewhere could easily flip which method scores marginally higher — the takeaway is the general pattern (adaptive search scales better to larger spaces) demonstrated through a controlled comparison methodology, not a universal ranking of the three methods.

Q: Why does the concept of parameter importance from a hyperparameter study parallel the concept of feature importance from a trained model, despite operating on entirely different kinds of 'inputs' (hyperparameters vs data features)?

A: Both answer a structurally identical question — 'how much does varying this input actually change the outcome, holding a model of the underlying relationship fixed' — just applied at different levels: feature importance operates within a single trained model's decision process, while hyperparameter importance operates across an entire study's worth of trained models to explain variance in their aggregate performance.

Key Takeaway and What's Next

Key takeaway: GridSearchCV, RandomizedSearchCV, and Optuna form a natural progression, not competing answers: exhaustive-and-narrow, wide-and-blind, and wide-and-adaptive. Verified today at an identical 90-fit budget, Optuna's adaptive sampling narrowly outperformed both alternatives while searching more than double the grid's hyperparameters — an advantage expected to grow further at larger trial budgets than this lesson used.

What typically follows hyperparameter optimization

- Model persistence and serving — saving the tuned pipeline (and, ideally, the Optuna study itself) with joblib, so the tuning work doesn't need to be repeated and the model can be served behind an API.
- Deep learning foundations — the exact same search strategies (grid, random, Bayesian) reappear when tuning a neural network's learning rate, layer widths, and regularization, reusing everything learned across Week 5 without modification.
- Week 5 review — a recap tying interpretability (Day 28-30) and optimization (Day 31-32) together as the two halves of understanding and improving a model once it's already trained.